

selects the projection matrices where LoRA adapters are inserted, choosing attention (q/k/v/o) guides context routing, while choosing MLP (gate/up/down) shapes token-wise transformations. Together, these enable strong, parameter-efficient adaptation during fine-tuning.

lora_dropout=0.05: This randomly disables 5% of LoRA adapter activations during training to prevent overfitting, stabilise optimisation, and ensure the low-rank updates generalise well without overspecialising to the fine-tuning dataset.

optim=(“paged_adamw_8bit” if USE_4BIT else “adamw_torch”): In LLM fine-tuning, the optimiser is the algorithm that updates LoRA adapter parameters according to gradients, controlling how fast, how stable, and how effectively the model learns from the training data. For our case we have used the adamw_8bit optimiser function provided by bitsandbytes library. This is suited for QLoRA and free tier T4 GPU VRAM requirement.

Max_grad_norm=1.0: This enables gradient clipping so that the total gradient magnitude never exceeds 1.0, preventing unstable updates and ensuring smooth, safe LoRA/QLoRA fine-tuning of large language models.

Step 6: Model training (SFTTrainer execution)

`trainer.train()` runs the full supervised fine-tuning loop, updating only LoRA adapter weights using the optimiser and training strategy defined earlier. After training, `model.save_pretrained()` stores the learned LoRA updates and configuration, while `tokenizer.save_pretrained()` saves the tokenization rules so the fine-tuned model can be correctly reloaded for inference or further training.

```
trainer.train()
```

```
trainer.model.save_pretrained(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)
```

Step 7: Apply trained adapters and quick inferencing

This piece of code applies trained adapters on the base model (Mistral 7B) and runs the quick inferencing. Some important parts of the code are explained below.

```
from transformers import AutoTokenizer, AutoModelForCausalLM,
BitsAndBytesConfig
from peft import PeftModel
from peft import prepare_model_for_kbit_training
import torch
```

```
BASE_MODEL_NAME = BASE_MODEL
LORA_DIR = OUTPUT_DIR
```

```
bnb_config = BitsAndBytesConfig(
```

```
    load_in_4bit=USE_4BIT,
    bnb_4bit_quant_type='nf4',
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.float16,
)
```

```
tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL, use_
fast=True)
tokenizer.padding_side = "right"
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
```

```
base_model = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL_NAME,
    quantization_config=bnb_config,
    device_map={"": 0},
)
base_model = prepare_model_for_kbit_training(base_model)
```

```
infer_model = PeftModel.from_pretrained(
    base_model,
    LORA_DIR,
    is_trainable=False,
)
```

```
infer_model.eval()
infer_model.config.use_cache = True
infer_model.config.pad_token_id = tokenizer.pad_token_id
infer_model.config.eos_token_id = tokenizer.eos_token_id
infer_model.config.bos_token_id = tokenizer.bos_token_id
```

```
def generate(prompt, max_new_tokens=300):
    inputs = tokenizer(prompt, return_tensors="pt").to(infer_
model.device)
```

```
    with torch.no_grad():
        output = infer_model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
```

```
            do_sample=True,
            temperature=0.7,
            top_p=0.9,
            top_k=50,
            repetition_penalty=1.1,

            pad_token_id=tokenizer.pad_token_id,
            eos_token_id=tokenizer.eos_token_id,
```

```
)
```